

R Code Examples from WDDA Lecture 5 (Chapter 5)

WDDA – Data Management and Data Analysis

Prof. Dr. Ulrich Matter

2026-04-29

Contents

1	Introduction	1
2	Preparation	2
3	Variation	2
4	Explanatory Variable	4
5	Comparing Candidates	5
6	Quantifying Deviations from the Model	6
7	Best Fit: Estimated Regression Line	8
8	Residuals	11
9	TSS and RSS	14
10	Decomposition of Variation	15
11	Coefficient of Determination	16
12	Expectations for Residuals	18
13	Prediction	21
14	Summary	23

1 Introduction

This document collects all R code examples from the WDDA FS2026 Lecture 5 slides (Chapter 5). The focus is on simple linear regression, a fundamental statistical method for modelling the relationship between two variables.

Simple linear regression is a statistical procedure that models the linear relationship between a dependent variable (target) and an independent variable (predictor). The goal is to find a line that best describes the relationship between the two variables.

Key concepts in this area are:

1. **Variation and covariance:** How much do the variables vary, and how are they related?

2. **Method of least squares:** How do we find the “best” regression line?
3. **Residuals:** How large are the deviations between observed and predicted values?
4. **Coefficient of determination (R^2):** How well does the model explain the variation in the data?
5. **Prediction:** How can we use the model for forecasts?

2 Preparation

```
# Load packages
library(readxl)

# Load data
data <- read_excel("data/WDDA_05.xlsx", sheet = "Advertising")

# Make variables directly accessible
attach(data)

## tibble [200 x 4] (S3: tbl_df/tbl/data.frame)
##   $ TV          : num [1:200] 230.1 44.5 17.2 151.5 180.8 ...
##   $ radio       : num [1:200] 37.8 39.3 45.9 41.3 10.8 48.9 32.8 19.6 2.1 2.6 ...
##   $ newspaper: num [1:200] 69.2 45.1 69.3 58.5 58.4 75 23.5 11.6 1 21.2 ...
##   $ sales      : num [1:200] 22.1 10.4 9.3 18.5 12.9 7.2 11.8 13.2 4.8 10.6 ...
```

R explanation:

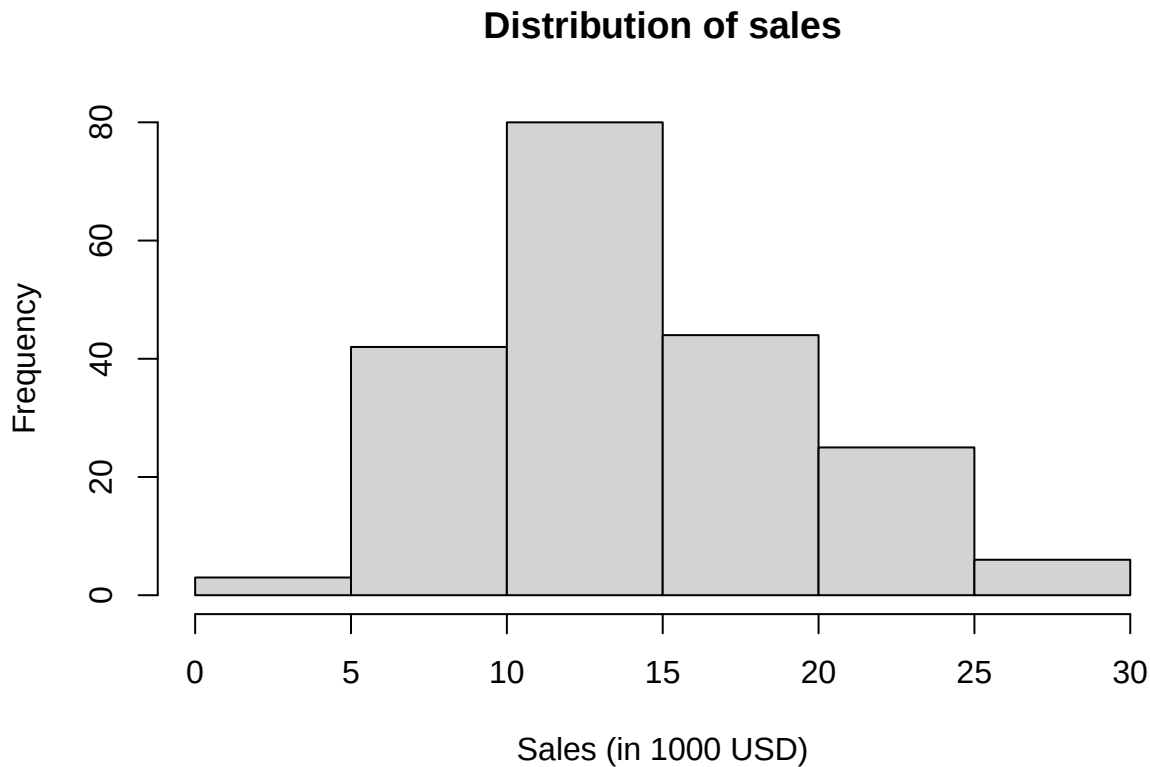
- The function `library()` loads an installed package into the current R session.
- With `read_excel()` from the `readxl` package, Excel files can be imported. The `sheet` parameter specifies which worksheet to use.
- The function `attach()` makes the variables of a data frame directly accessible without having to prefix the data frame name (e.g. `sales` instead of `data$sales`).
- The function `str()` shows the structure of an object, including data types and dimensions.

Note on data paths: In R Markdown files, relative paths are interpreted from the location of the file, hence the path `"../data/WDDA_05.xlsx"`. In regular R scripts, the path would be specified relative to the working directory.

3 Variation

Before we begin with the regression, we look at the variation in our target variable (sales).

```
# Basic visualisation: target variable
hist(sales, main = "Distribution of sales", xlab = "Sales (in 1000 USD)")
```



```
cat("Variance of sales:", round(var(sales), 2), "\n")
```

```
## Variance of sales: 27.22
```

```
# Additional statistics
```

```
cat("Mean of sales:", round(mean(sales), 2), "\n")
```

```
## Mean of sales: 14.02
```

```
cat("Standard deviation of sales:", round(sd(sales), 2), "\n")
```

```
## Standard deviation of sales: 5.22
```

```
cat("Minimum of sales:", min(sales), "\n")
```

```
## Minimum of sales: 1.6
```

```
cat("Maximum of sales:", max(sales), "\n")
```

```
## Maximum of sales: 27
```

Statistical explanation:

- The **variance** is a measure of the spread of the data around its mean.
- A high variance means that the values are widely spread around the mean.
- The variance is computed as the average of the squared deviations from the mean: $\text{Var}(X) = \frac{\sum (x_i - \bar{x})^2}{n}$
- The **standard deviation** is the square root of the variance and has the advantage that it is measured

in the same unit as the data itself.

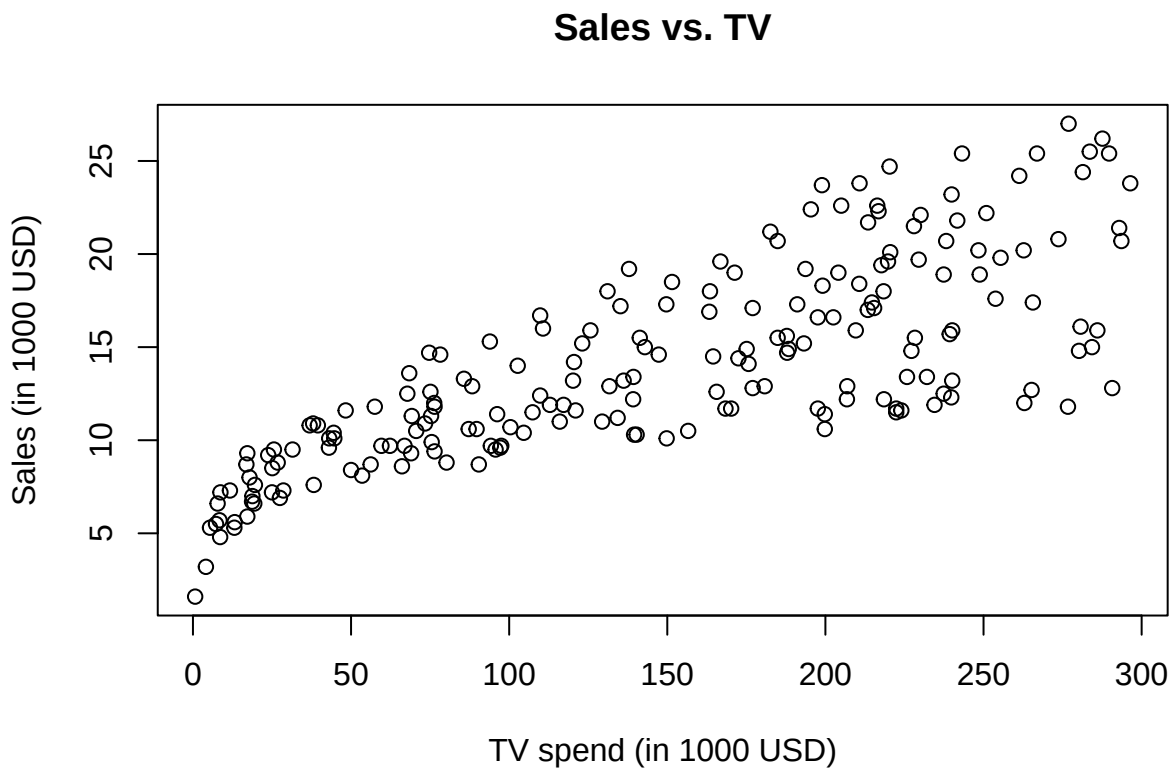
R explanation:

- The function `hist()` creates a histogram showing the frequency distribution of the data.
- The function `var()` computes the variance of a vector.
- The function `mean()` computes the mean of a vector.
- The function `sd()` computes the standard deviation of a vector.
- The functions `min()` and `max()` return the smallest and largest value of a vector, respectively.

4 Explanatory Variable

We now look at the relationship between our target variable (sales) and the explanatory variable (TV advertising spend).

```
# Scatterplot: relationship between TV and sales
plot(sales ~ TV, main = "Sales vs. TV", xlab = "TV spend (in 1000 USD)", ylab = "Sales
  ↪ (in 1000 USD)")
```



```
# Compute correlation
correlation <- cor(sales, TV)
cat("Correlation between sales and TV:", round(correlation, 3), "\n")
```

```
## Correlation between sales and TV: 0.782
```

```
# Compute covariance
covariance <- cov(sales, TV)
cat("Covariance between sales and TV:", round(covariance, 3), "\n")
```

```
## Covariance between sales and TV: 350.39
```

Statistical explanation:

- A **scatterplot** shows the relationship between two continuous variables.
- Each point represents an observation with the corresponding values of both variables.
- The **correlation** is a standardised measure of the linear relationship between two variables and lies between -1 and +1.
 - A correlation close to +1 indicates a strong positive linear relationship.
 - A correlation close to -1 indicates a strong negative linear relationship.
 - A correlation close to 0 indicates no linear relationship.
- The **covariance** is an unstandardised measure of the linear relationship between two variables.
 - The covariance has no fixed scale and depends on the units of the variables.
 - A positive covariance means that the variables tend to rise or fall together.
 - A negative covariance means that one variable tends to rise when the other falls.

R explanation:

- The formula `sales ~ TV` in `plot()` specifies that `sales` should be plotted on the y-axis and `TV` on the x-axis.
- The function `cor()` computes the Pearson correlation between two vectors.
- The function `cov()` computes the covariance between two vectors.

5 Comparing Candidates

We compare different possible regression lines to see which one best describes the relationship between TV advertising spend and sales.

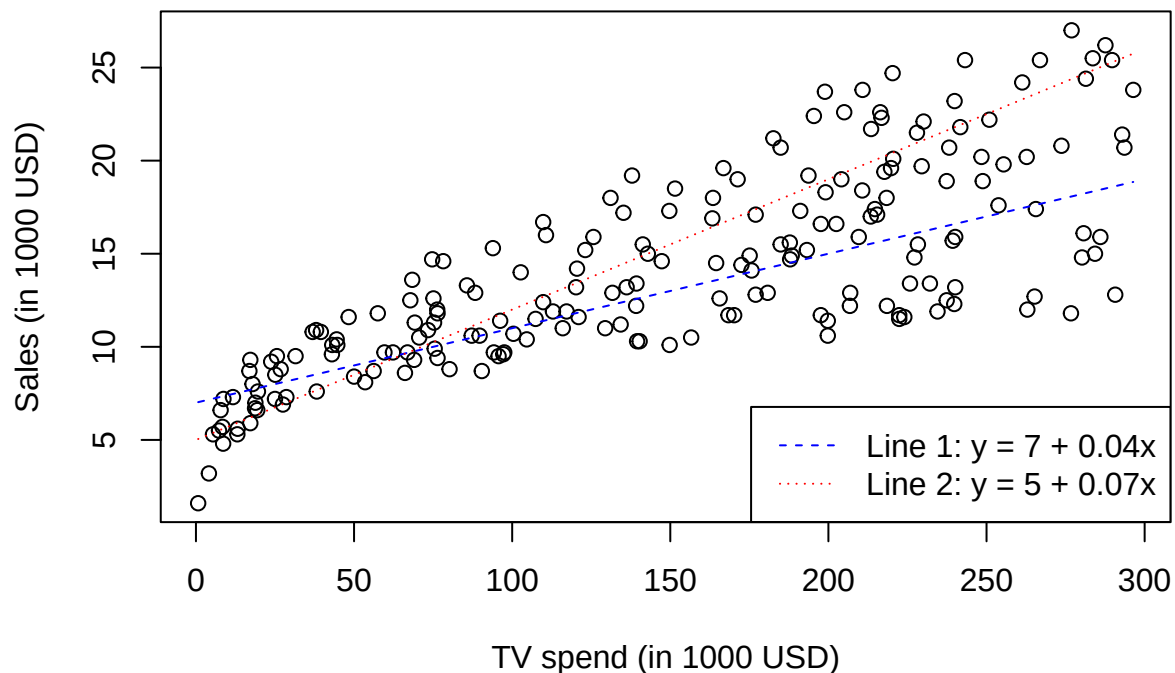
```
# Create scatterplot
plot(sales ~ TV, main = "Sales vs. TV - comparison of regression lines",
     xlab = "TV spend (in 1000 USD)", ylab = "Sales (in 1000 USD)")

# Two possible lines (manually defined)
line1 <- function(x) { 7 + 0.04 * x }
line2 <- function(x) { 5 + 0.07 * x }

# Add lines to the plot
curve(line1, add = TRUE, col = "blue", lty = 2)
curve(line2, add = TRUE, col = "red", lty = 3)

# Add a legend
legend("bottomright",
     legend = c("Line 1: y = 7 + 0.04x", "Line 2: y = 5 + 0.07x"),
     col = c("blue", "red"),
     lty = c(2, 3))
```

Sales vs. TV – comparison of regression lines



Statistical explanation:

- A **regression line** has the form $y = b + b x$, where:
 - b is the y-intercept: the predicted value of y when $x = 0$.
 - b is the slope: the change in y associated with a one-unit change in x .
- Different lines can describe the relationship between two variables to different degrees.
- The “best” line is the one that minimises the deviations between observed and predicted values.

R explanation:

- The function `curve()` with the parameter `add = TRUE` adds a curve to an existing plot.
- The parameter `lty` (line type) in `curve()` and `legend()` specifies the line type (1 = solid, 2 = dashed, 3 = dotted, etc.).
- The function `legend()` adds a legend to the plot.

6 Quantifying Deviations from the Model

To determine which regression line is better, we quantify the deviations between observed and predicted values.

```
# Compute predicted values for both lines
pred_line1 <- line1(TV)
pred_line2 <- line2(TV)

# Compute residuals (deviations)
resid_line1 <- sales - pred_line1
resid_line2 <- sales - pred_line2

# Compute squared residuals
```

```

sq_resid_line1 <- resid_line1^2
sq_resid_line2 <- resid_line2^2

# Compute the residual sum of squares (RSS)
rss_line1 <- sum(sq_resid_line1)
rss_line2 <- sum(sq_resid_line2)

# Print results
cat("Sum of Squared Errors (RSS):\n")

## Sum of Squared Errors (RSS):

cat("Line 1 (y = 7 + 0.04x):", round(rss_line1, 2), "\n")

## Line 1 (y = 7 + 0.04x): 2446.13

cat("Line 2 (y = 5 + 0.07x):", round(rss_line2, 2), "\n")

## Line 2 (y = 5 + 0.07x): 3165.51

# Visualise the residuals for the first observation
plot(sales ~ TV, main = "Residuals for the first observation",
     xlab = "TV spend (in 1000 USD)", ylab = "Sales (in 1000 USD)")
curve(line1, add = TRUE, col = "blue", lty = 2)
curve(line2, add = TRUE, col = "red", lty = 3)

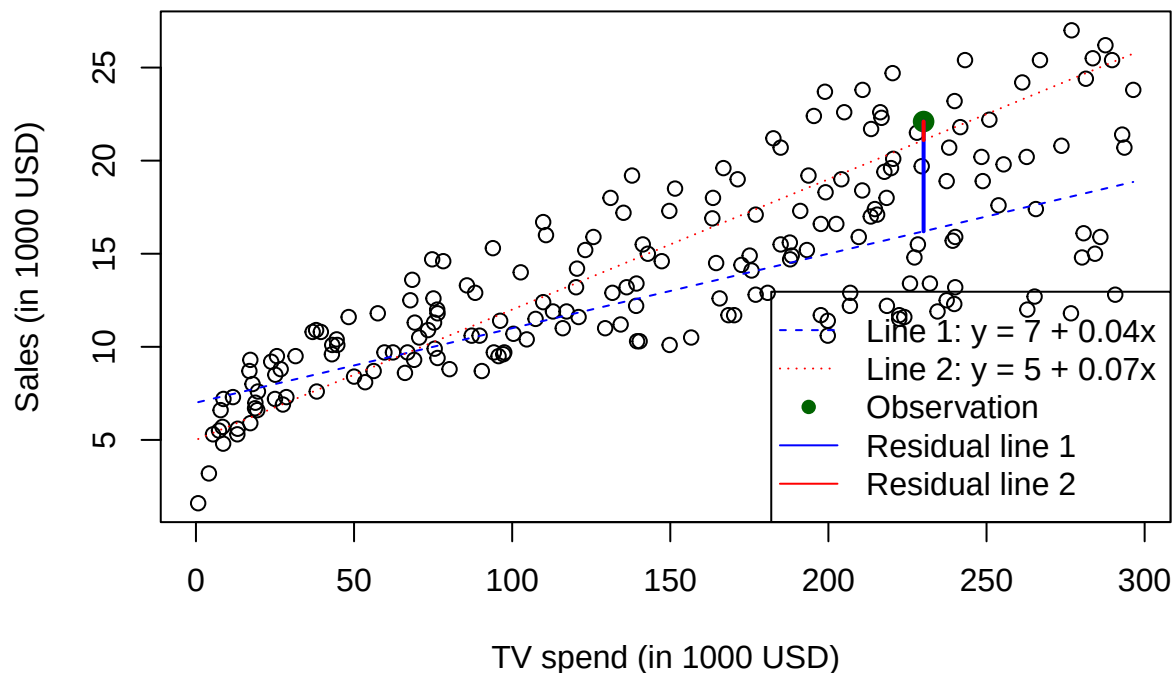
# Highlight the first observation
points(TV[1], sales[1], col = "darkgreen", pch = 16, cex = 1.5)

# Draw residuals for the first observation
segments(TV[1], sales[1], TV[1], pred_line1[1], col = "blue", lwd = 2)
segments(TV[1], sales[1], TV[1], pred_line2[1], col = "red", lwd = 2)

# Add a legend
legend("bottomright",
      legend = c("Line 1: y = 7 + 0.04x", "Line 2: y = 5 + 0.07x", "Observation",
        ↪ "Residual line 1", "Residual line 2"),
      col = c("blue", "red", "darkgreen", "blue", "red"),
      lty = c(2, 3, NA, 1, 1),
      pch = c(NA, NA, 16, NA, NA))

```

Residuals for the first observation



Statistical explanation:

- **Residuals** are the differences between observed and predicted values: $e_i = y_i - \hat{y}_i$
- The **residual sum of squares** (RSS) is a measure of the overall deviation of the model: $RSS = \sum (y_i - \hat{y}_i)^2$
- The smaller the RSS, the better the model fits the data.
- The **method of least squares** chooses the regression line that minimises the RSS.
- In this example, line 1 ($y = 7 + 0.04x$) has a smaller RSS than line 2 ($y = 5 + 0.07x$), suggesting that it fits the data better.

R explanation:

- The operator `^2` computes the square of a vector.
- The function `sum()` computes the sum of the elements of a vector.
- The function `segments()` draws line segments between two points.
- The parameter `pch` (plotting character) in `points()` and `legend()` specifies the symbol for points.
- The parameter `cex` (character expansion) in `points()` specifies the size of the points.

7 Best Fit: Estimated Regression Line

We now compute the optimal regression line according to the method of least squares.

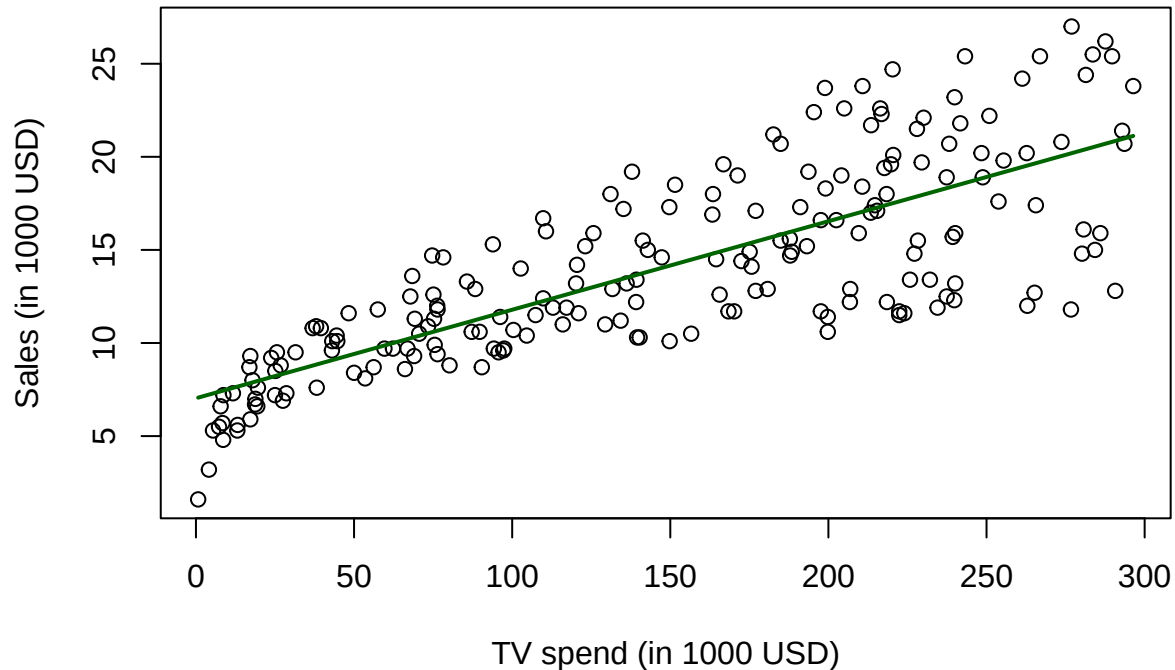
```
# Method of least squares (manually)
b1 <- cor(sales, TV) * sd(sales) / sd(TV)
b0 <- mean(sales) - b1 * mean(TV)
lbf <- function(x) { b0 + b1 * x }

# Draw the fitted line
plot(sales ~ TV, main = "Best fit line (least squares)",
```



```
xlab = "TV spend (in 1000 USD)", ylab = "Sales (in 1000 USD)"
curve(lbf(x), add = TRUE, col = "darkgreen", lwd = 2)
```

Best fit line (least squares)



```
# Compute deviation (RSS)
pred_lbf <- lbf(TV)
resid_lbf <- sales - pred_lbf
rss <- sum(resid_lbf^2)
cat("Coefficients of the fitted line:\n")
```

```
## Coefficients of the fitted line:
```

```
cat("Intercept (b):", round(b0, 4), "\n")
```

```
## Intercept (b): 7.0326
```

```
cat("Slope (b):", round(b1, 4), "\n")
```

```
## Slope (b): 0.0475
```

```
cat("Minimum deviation (RSS) of the fitted line:", round(rss, 2), "\n")
```

```
## Minimum deviation (RSS) of the fitted line: 2102.53
```

```
# Regression model with lm()
mdl <- lm(sales ~ TV)
summary(mdl)
```

```
##
## Call:
## lm(formula = sales ~ TV)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -8.3860 -1.9545 -0.1913  2.0671  7.2124
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  7.032594   0.457843   15.36  <2e-16 ***
## TV           0.047537   0.002691   17.67  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.259 on 198 degrees of freedom
## Multiple R-squared:  0.6119, Adjusted R-squared:  0.6099
## F-statistic: 312.1 on 1 and 198 DF,  p-value: < 2.2e-16
```

```
# Compare manual computation with lm()
cat("\nComparison of coefficients:\n")
```

```
##
## Comparison of coefficients:
```

```
cat("Manual:  y =", round(b0, 4), "+", round(b1, 4), "* x\n")
```

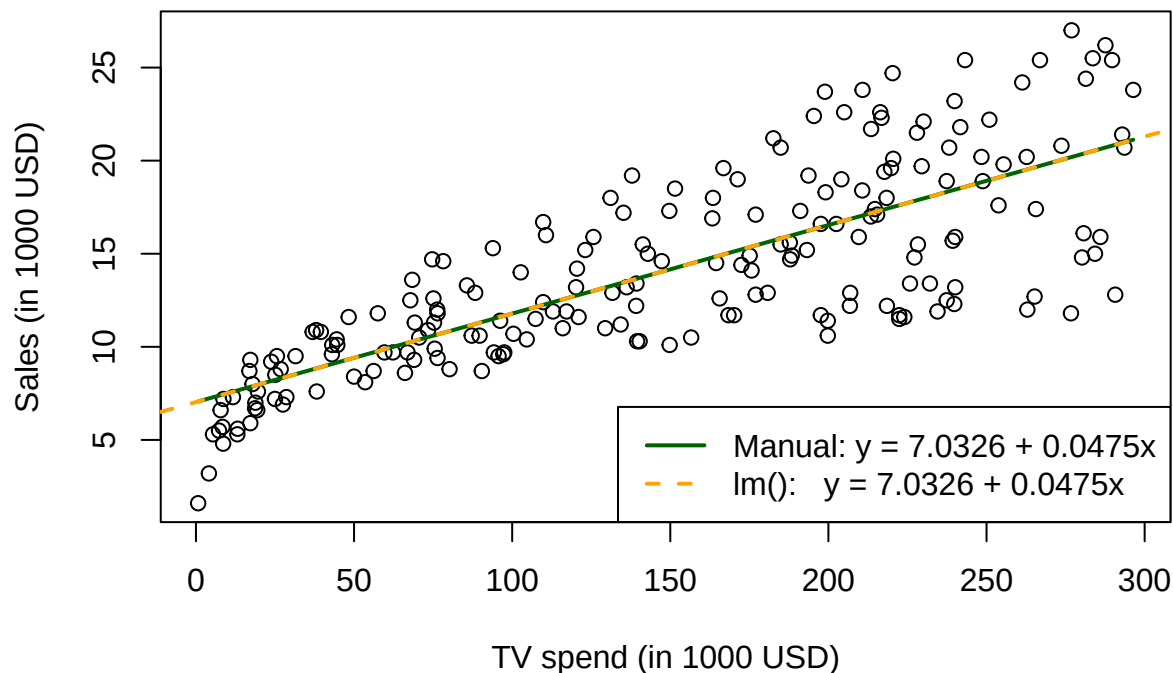
```
## Manual:  y = 7.0326 + 0.0475 * x
```

```
cat("With lm(): y =", round(coef(md1)[1], 4), "+", round(coef(md1)[2], 4), "* x\n")
```

```
## With lm(): y = 7.0326 + 0.0475 * x
```

```
# Show both lines in the plot
plot(sales ~ TV, main = "Comparison: manual computation vs. lm()",
      xlab = "TV spend (in 1000 USD)", ylab = "Sales (in 1000 USD)")
curve(lbf(x), add = TRUE, col = "darkgreen", lwd = 2)
abline(md1, col = "orange", lwd = 2, lty = 2)
legend("bottomright",
      legend = c(sprintf("Manual: y = %.4f + %.4fx", b0, b1),
                  sprintf("lm():  y = %.4f + %.4fx", coef(md1)[1], coef(md1)[2])),
      col = c("darkgreen", "orange"),
      lty = c(1, 2),
      lwd = 2)
```

Comparison: manual computation vs. lm()



Statistical explanation:

- The **method of least squares** yields the following formulas for the coefficients:
 - $b = \text{Cov}(X, Y) / \text{Var}(X) = r * (s_Y / s_X)$, where r is the correlation between X and Y .
 - $b = Y - b * X$
- These formulas guarantee that the resulting line minimises the residual sum of squares (RSS).
- The function `lm()` in R implements the method of least squares and provides additional information about the regression model.
- The manual computation and `lm()` should yield identical coefficients.

R explanation:

- The function `lm()` (linear model) fits a linear regression model to the data.
- The formula `sales ~ TV` in `lm()` specifies that `sales` is the dependent variable and `TV` is the independent variable.
- The function `summary()` provides a detailed summary of the regression model, including coefficients, standard errors, t-values, p-values, and R^2 .
- The function `coef()` extracts the coefficients from a regression model.
- The function `abline()` adds a line to an existing plot, based on a regression model.

8 Residuals

We now examine the residuals of the regression model in order to assess the quality of the fit.

```
# 1st residual (observation: TV = 230.1, sales = 22.1)
tv_val <- 230.1
sales_obs <- 22.1

# Compute predicted value
```

```
sales_pred <- b0 + b1 * tv_val
cat("Predicted value at TV =", tv_val, ":", round(sales_pred, 2), "\n")
```

```
## Predicted value at TV = 230.1 : 17.97
```

```
# Compute residual
resid1 <- sales_obs - sales_pred
cat("1st residual:", round(resid1, 2), "\n")
```

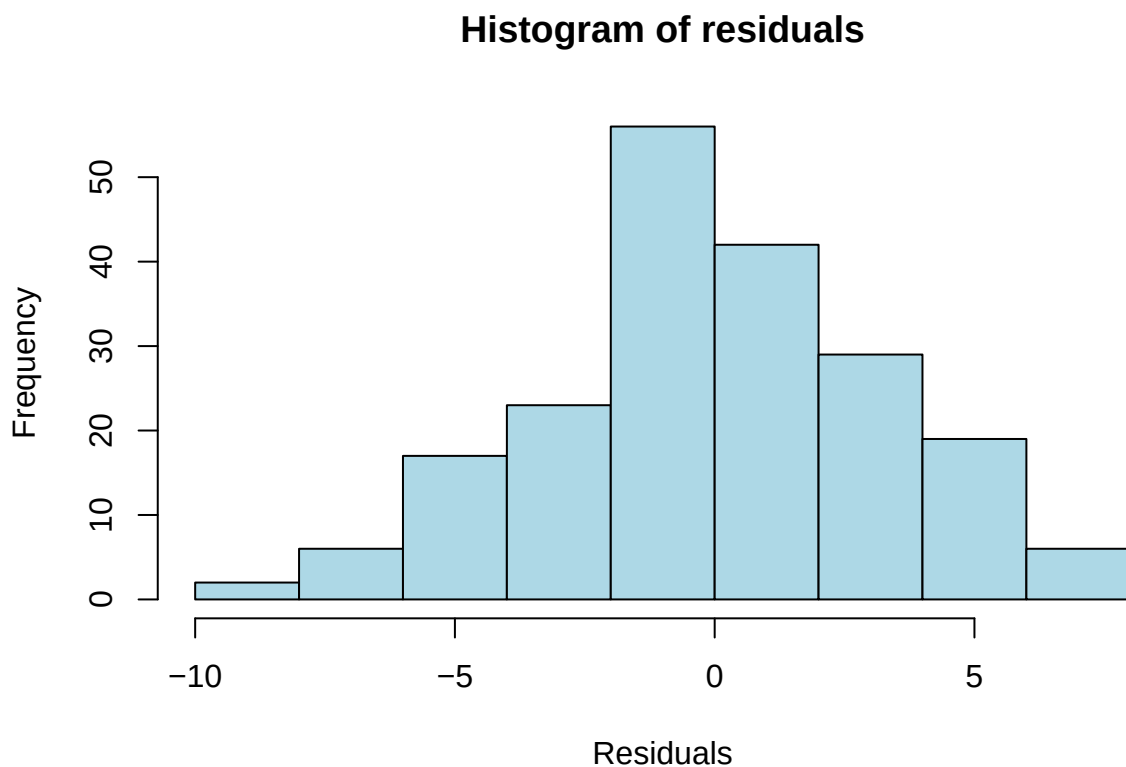
```
## 1st residual: 4.13
```

```
# Alternative computation with predict()
sales_hat <- predict(md1, newdata = data.frame(TV = tv_val))
resid1_alt <- sales_obs - sales_hat
cat("1st residual (using predict()):", round(resid1_alt, 2), "\n")
```

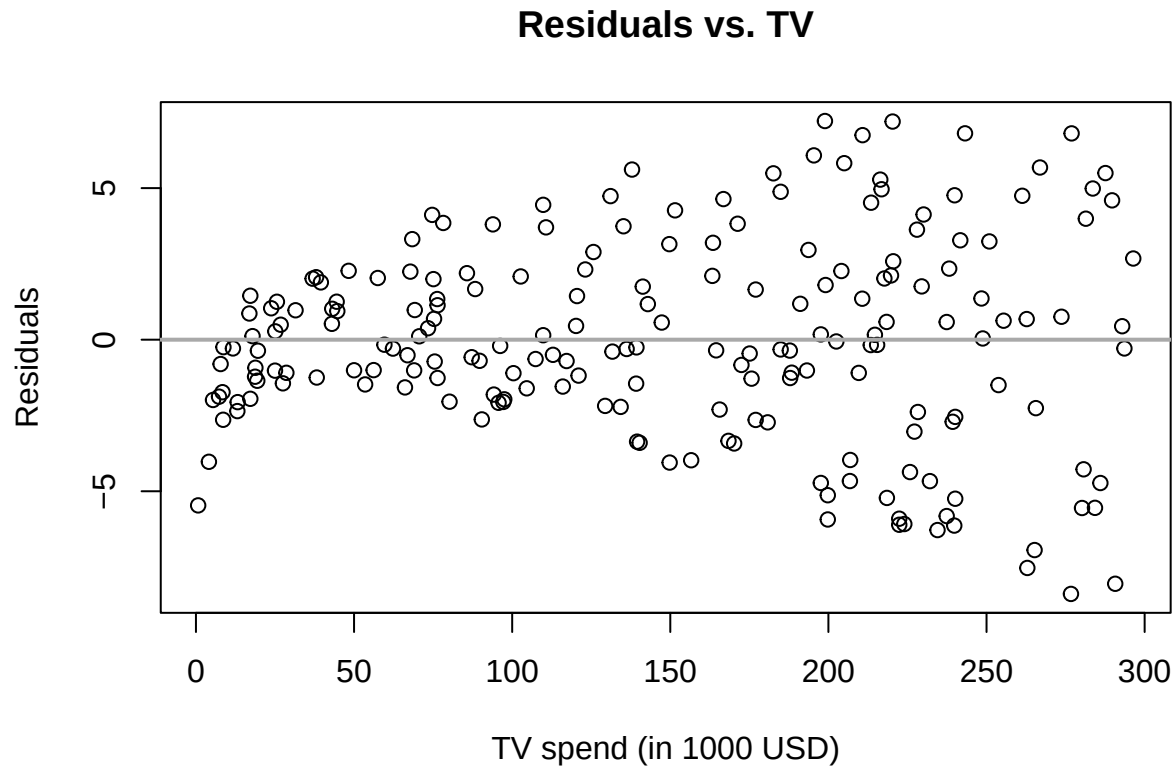
```
## 1st residual (using predict()): 4.13
```

```
# Compute all residuals
residuals_all <- resid(md1)
```

```
# Histogram of residuals
hist(residuals_all,
     main = "Histogram of residuals",
     xlab = "Residuals",
     col = "lightblue")
```



```
# Residuals vs. predictor
plot(residuals_all ~ TV,
     main = "Residuals vs. TV",
     xlab = "TV spend (in 1000 USD)",
     ylab = "Residuals")
abline(h = 0, col = "darkgray", lwd = 2)
```



Statistical explanation:

- **Residuals** are the differences between observed and predicted values: $e_i = y_i - \hat{y}_i$
- The residuals provide insight into the quality of the model fit:
 - If the residuals are randomly distributed around zero, the model is appropriate.
 - If the residuals exhibit patterns, the model could be improved.
- Residual analysis is an important step in checking the assumptions of linear regression:
 - Normality: the residuals should be normally distributed.
 - Homoscedasticity: the variance of the residuals should be constant.
 - Independence: the residuals should be independent of each other.
 - Linearity: the relationship between X and Y should be linear.

R explanation:

- The function `predict()` computes predicted values for a regression model.
- The parameter `newdata` in `predict()` specifies a data frame with the values of the predictors for which predictions should be made.
- The function `resid()` extracts the residuals from a regression model.
- The function `abline()` with the parameter `h` adds a horizontal line to the plot.

9 TSS and RSS

We now examine the decomposition of the total variation into explained and unexplained variation.

```
# Sums of squares: TSS, RSS, ESS
TSS <- sum((sales - mean(sales))^2)
RSS <- sum((sales - lbf(TV))^2)
ESS <- sum((lbf(TV) - mean(lbf(TV)))^2)

# Alternative computation of ESS
ESS_alt <- sum((lbf(TV) - mean(sales))^2)

# Print sums of squares
cat("TSS (total variation):", round(TSS, 2), "\n")

## TSS (total variation): 5417.15

cat("RSS (residuals):", round(RSS, 2), "\n")

## RSS (residuals): 2102.53

cat("ESS (explained variation):", round(ESS, 2), "\n")

## ESS (explained variation): 3314.62

cat("ESS (alternative computation):", round(ESS_alt, 2), "\n")

## ESS (alternative computation): 3314.62

cat("TSS = ESS + RSS:", round(TSS, 2), "=", round(ESS, 2), "+", round(RSS, 2), "\n")

## TSS = ESS + RSS: 5417.15 = 3314.62 + 2102.53

cat("Difference (should be close to 0):", round(TSS - (ESS + RSS), 10), "\n")

## Difference (should be close to 0): 0
```

Statistical explanation:

- The **total sum of squares** (TSS) measures the total variation in the dependent variable: $TSS = \sum(y_i - \bar{y})^2$
- The **residual sum of squares** (RSS) measures the unexplained variation: $RSS = \sum(y_i - \hat{y}_i)^2$
- The **explained sum of squares** (ESS) measures the variation explained by the model: $ESS = \sum(\hat{y}_i - \bar{y})^2$
- The decomposition holds: $TSS = ESS + RSS$
- This decomposition is the basis for the coefficient of determination R^2 .

R explanation:

- TSS, RSS, and ESS are computed directly from their respective formulas.
- The function `mean()` computes the mean of a vector.
- The function `round()` rounds a number to a specified number of decimal places.

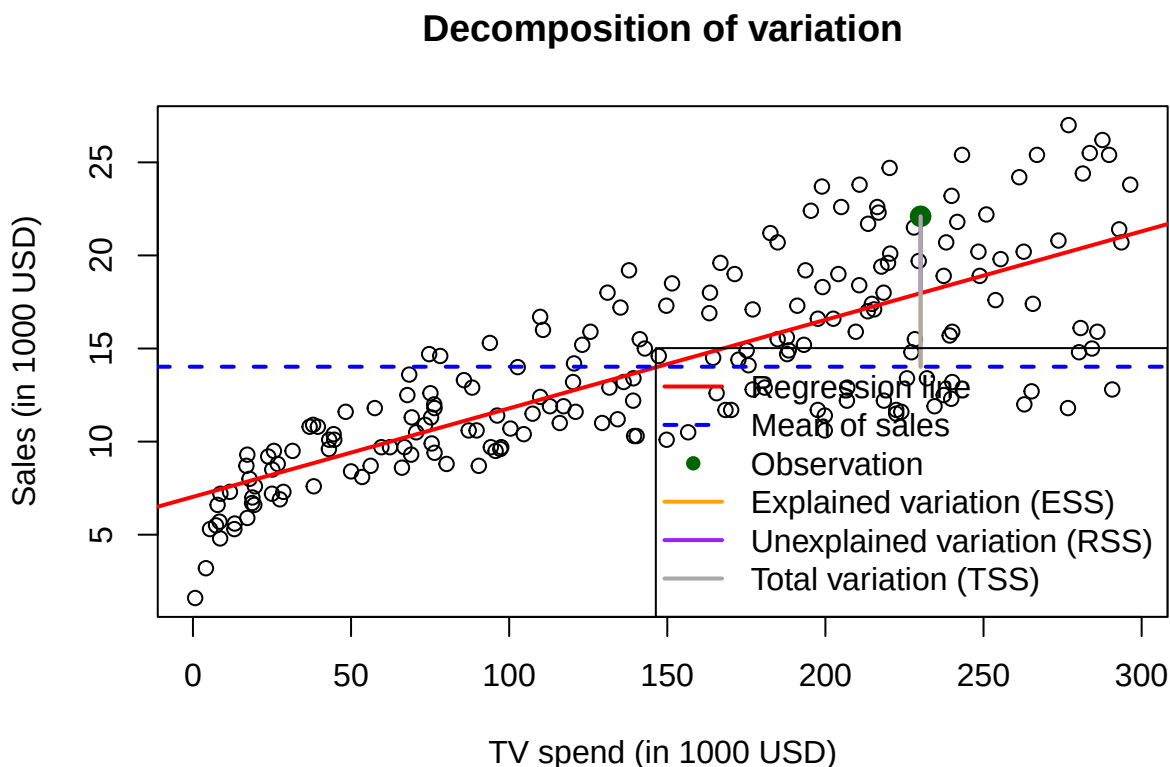
10 Decomposition of Variation

We visualise the decomposition of the total variation into explained and unexplained variation.

```
# Visualise the decomposition of variation
plot(sales ~ TV,
     main = "Decomposition of variation",
     xlab = "TV spend (in 1000 USD)",
     ylab = "Sales (in 1000 USD)")
abline(mdl, col = "red", lwd = 2)
abline(h = mean(sales), col = "blue", lwd = 2, lty = 2)

# For the first observation
points(TV[1], sales[1], col = "darkgreen", pch = 16, cex = 1.5)
segments(TV[1], mean(sales), TV[1], pred_lbf[1], col = "orange", lwd = 2) # ESS
segments(TV[1], pred_lbf[1], TV[1], sales[1], col = "purple", lwd = 2) # RSS
segments(TV[1], mean(sales), TV[1], sales[1], col = "darkgray", lwd = 2) # TSS

# Add a legend
legend("bottomright",
     legend = c("Regression line", "Mean of sales", "Observation",
               "Explained variation (ESS)", "Unexplained variation (RSS)", "Total
               ↪ variation (TSS)"),
     col = c("red", "blue", "darkgreen", "orange", "purple", "darkgray"),
     lty = c(1, 2, NA, 1, 1, 1),
     pch = c(NA, NA, 16, NA, NA, NA),
     lwd = 2)
```



```

# Print sums of squares
cat("Decomposition of variation:\n")

## Decomposition of variation:

cat("TSS (total variation):", round(TSS, 2), "\n")

## TSS (total variation): 5417.15

cat("RSS (residuals):", round(RSS, 2), "\n")

## RSS (residuals): 2102.53

cat("ESS (explained variation):", round(ESS, 2), "\n")

## ESS (explained variation): 3314.62

cat("TSS = ESS + RSS:", round(TSS, 2), "=", round(ESS, 2), "+", round(RSS, 2), "\n")

## TSS = ESS + RSS: 5417.15 = 3314.62 + 2102.53

```

Statistical explanation:

- The **decomposition of variation** is a central concept in regression analysis:
 - The total variation (TSS) is decomposed into explained variation (ESS) and unexplained variation (RSS).
 - The explained variation (ESS) is the variation that is explained by the regression model.
 - The unexplained variation (RSS) is the variation that is not explained by the model (residuals).
- For each observation: $(y_i - \bar{y}) = (\hat{y}_i - \bar{y}) + (y_i - \hat{y}_i)$
 - $(y_i - \bar{y})$ is the total deviation (TSS component)
 - $(\hat{y}_i - \bar{y})$ is the explained deviation (ESS component)
 - $(y_i - \hat{y}_i)$ is the unexplained deviation (RSS component)

R explanation:

- The function `segments()` draws line segments between two points.
- The parameters `col`, `lwd`, and `lty` in `segments()` specify the colour, width, and type of the line.
- The function `points()` adds points to an existing plot.
- The function `legend()` adds a legend to the plot.

11 Coefficient of Determination

The coefficient of determination R^2 indicates how well the regression model explains the variation in the data.

```

# Coefficient of determination  $R^2$ 
r_squared <- (TSS - RSS) / TSS
r_squared_alt <- ESS / TSS
r_squared_check <- cor(sales, TV)^2

# Print the various computations of  $R^2$ 
cat("Coefficient of determination  $R^2$  (from variance analysis):", round(r_squared, 4),
    "\n")

```



```
## Coefficient of determination  $R^2$  (from variance analysis): 0.6119
```

```
cat("Coefficient of determination  $R^2$  (alternative computation):", round(r_squared_alt,
  ↪ 4), "\n")
```

```
## Coefficient of determination  $R^2$  (alternative computation): 0.6119
```

```
cat("Coefficient of determination  $R^2$  (from correlation):", round(r_squared_check, 4),
  ↪ "\n")
```

```
## Coefficient of determination  $R^2$  (from correlation): 0.6119
```

```
#  $R^2$  from the lm object
```

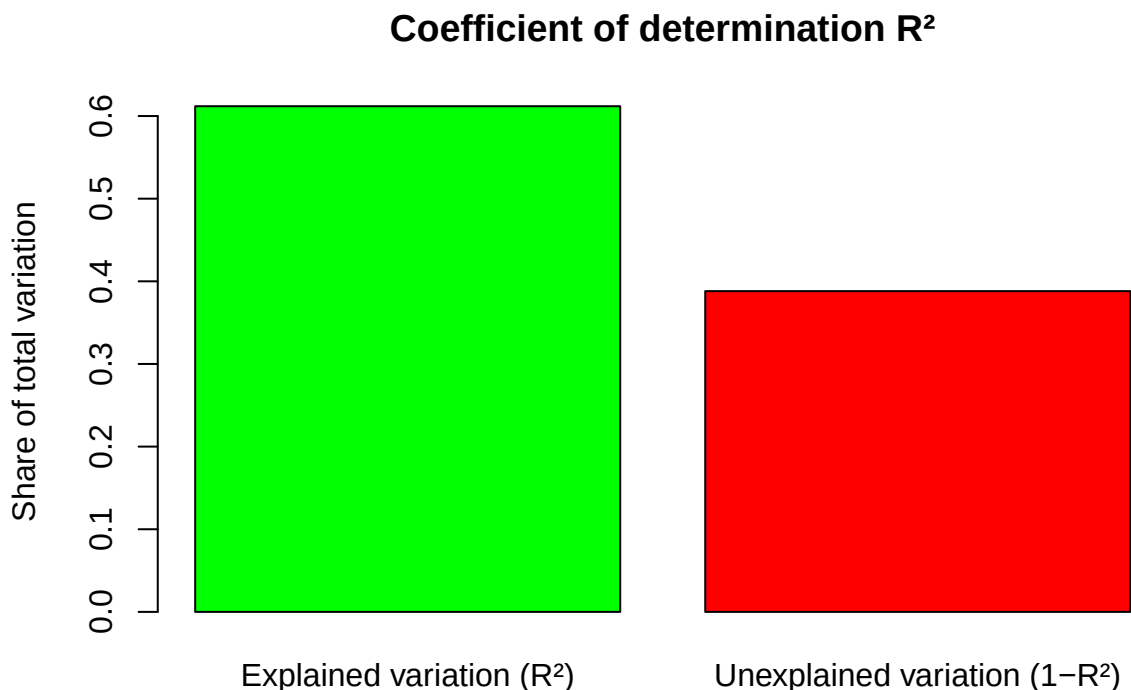
```
r_squared_lm <- summary(md1)$r.squared
```

```
cat("Coefficient of determination  $R^2$  (from lm()):", round(r_squared_lm, 4), "\n")
```

```
## Coefficient of determination  $R^2$  (from lm()): 0.6119
```

```
# Visualisation of  $R^2$ 
```

```
barplot(c(r_squared, 1 - r_squared),
  names.arg = c("Explained variation ( $R^2$ )", "Unexplained variation ( $1-R^2$ )"),
  col = c("green", "red"),
  main = "Coefficient of determination  $R^2$ ",
  ylab = "Share of total variation")
```



Statistical explanation:

- The **coefficient of determination R^2** indicates the share of variation in the dependent variable that is explained by the regression model.
- R^2 lies between 0 and 1:

- $R^2 = 0$: the model explains none of the variation in the data.
- $R^2 = 1$: the model explains all of the variation in the data.
- R^2 can be computed in several ways:
 - $R^2 = (\text{TSS} - \text{RSS}) / \text{TSS}$
 - $R^2 = \text{ESS} / \text{TSS}$
 - $R^2 = r^2$, where r is the correlation between X and Y (only for simple linear regression)
- A high R^2 means that the model fits the data well, but it does not guarantee that the model is causal or predictive.
- R^2 can be artificially inflated by adding more predictors, which is why the adjusted R^2 is often used in multiple regression.

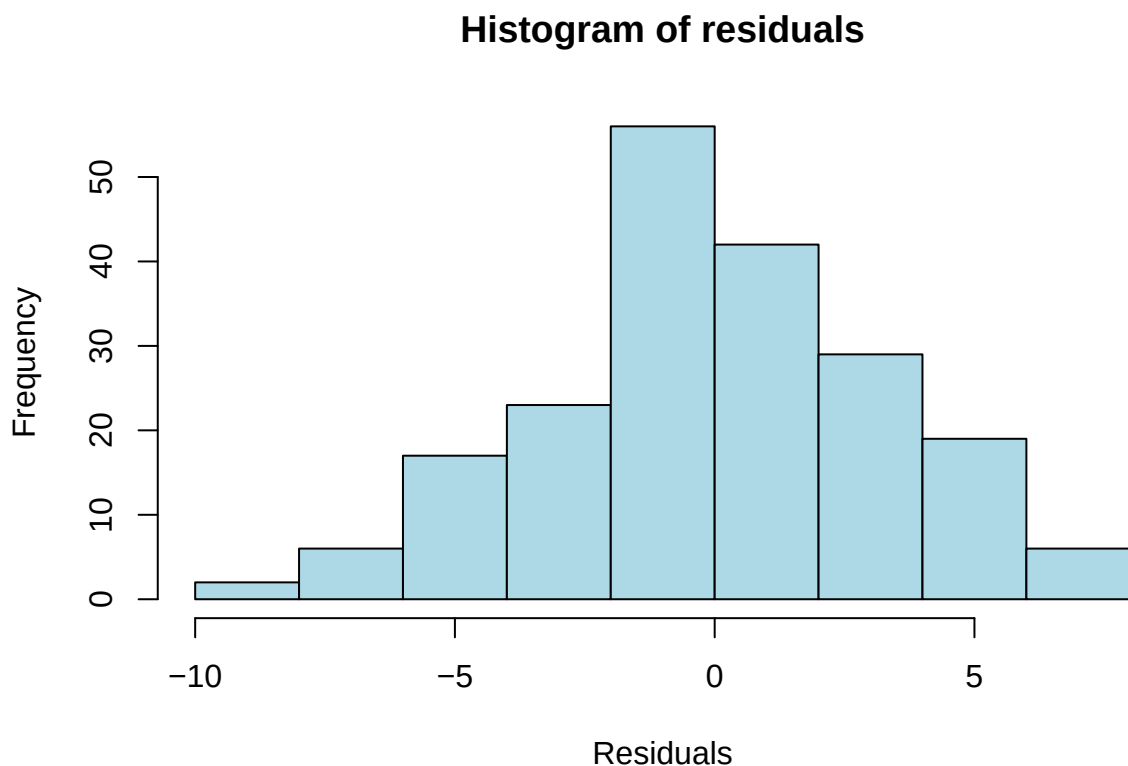
R explanation:

- The function `summary()` provides a detailed summary of the regression model, including R^2 .
- The expression `summary(md1)$r.squared` extracts the R^2 value from the regression object.
- The function `barplot()` creates a bar chart.
- The parameter `names.arg` in `barplot()` specifies the names of the bars.
- The parameter `col` in `barplot()` specifies the colours of the bars.

12 Expectations for Residuals

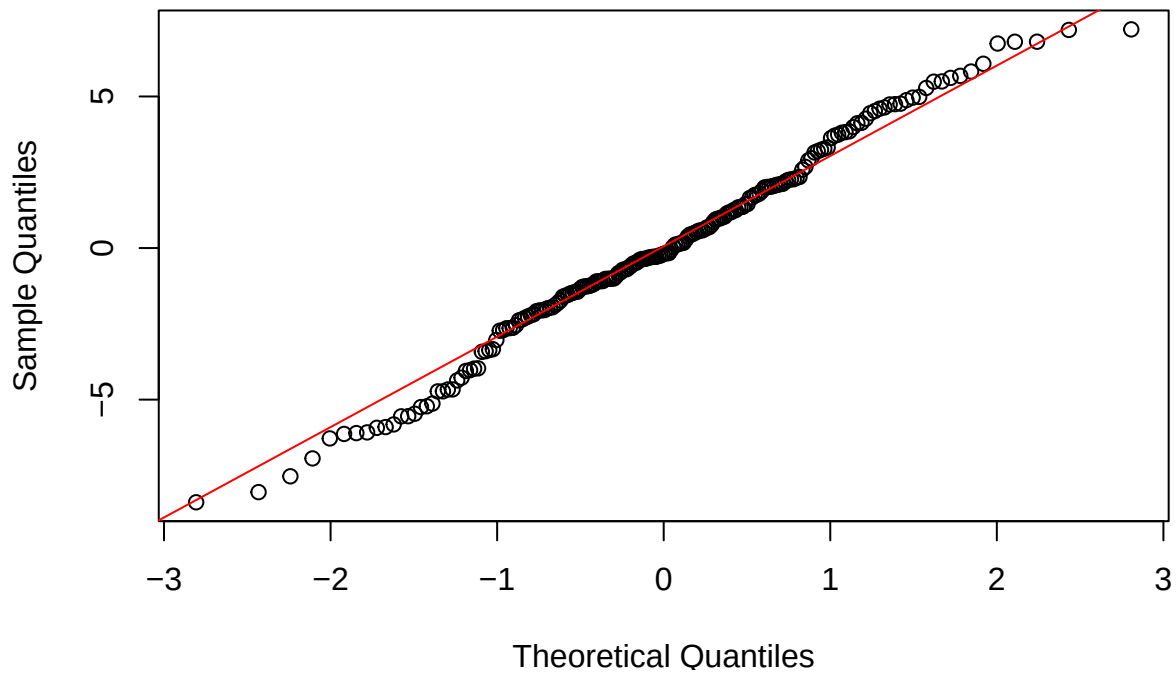
We examine the properties of the residuals in order to check the assumptions of linear regression.

```
# Residual diagnostics
hist(resid(md1),
     main = "Histogram of residuals",
     xlab = "Residuals",
     col = "lightblue")
```

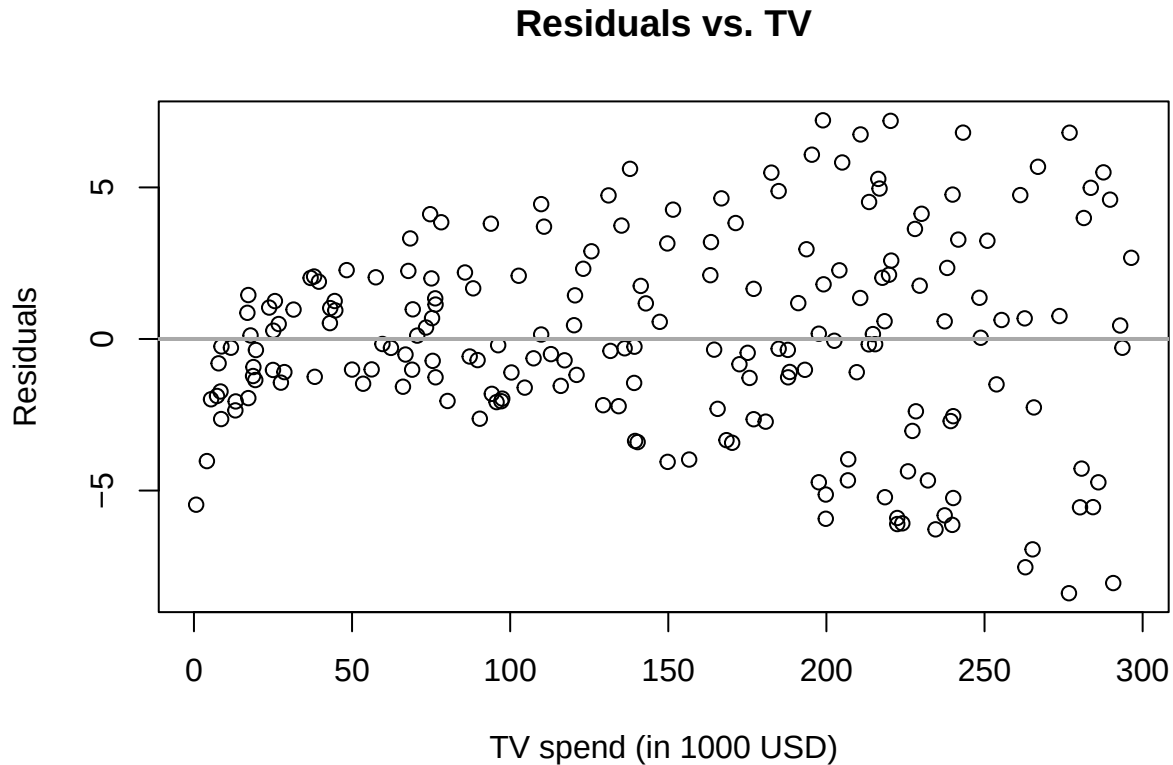


```
# Q-Q plot of residuals
qqnorm(resid(md1), main = "Q-Q plot of residuals")
qqline(resid(md1), col = "red")
```

Q-Q plot of residuals



```
# Residuals vs. predictor
plot(resid(md1) ~ TV,
     main = "Residuals vs. TV",
     xlab = "TV spend (in 1000 USD)",
     ylab = "Residuals")
abline(h = 0, col = "darkgray", lwd = 2)
```



```
# Residual standard error (RSE)
n <- length(sales)
RSE_manual <- sqrt(RSS / (n - 2))
RSE_lm <- summary(md1)$sigma
cat("RSE (manual):", round(RSE_manual, 3), "\n")
```

```
## RSE (manual): 3.259
```

```
cat("RSE (from lm()):", round(RSE_lm, 3), "\n")
```

```
## RSE (from lm()): 3.259
```

```
# Interpretation of the RSE
cat("Interpretation of the RSE:\n")
```

```
## Interpretation of the RSE:
```

```
cat("The typical error when predicting sales is about", round(RSE_lm, 3), "thousand
↳ USD.\n")
```

```
## The typical error when predicting sales is about 3.259 thousand USD.
```

Statistical explanation:

- The **assumptions of linear regression** mainly concern the properties of the residuals:
 - Normality: the residuals should be normally distributed.
 - Homoscedasticity: the variance of the residuals should be constant.
 - Independence: the residuals should be independent of each other.

- Linearity: the relationship between X and Y should be linear.
- The **residual standard error** (RSE) is a measure of the typical size of the residuals:
 - $RSE = \sqrt{(RSS / (n - 2))}$
 - The RSE has the same unit as the dependent variable.
 - A smaller RSE indicates a better fit of the model to the data.
- The RSE can be used for the construction of confidence and prediction intervals.

R explanation:

- The function `qqnorm()` creates a Q-Q plot that compares the distribution of the residuals with the normal distribution.
- The function `qqline()` adds a reference line to the Q-Q plot.
- The expression `summary(md1)$sigma` extracts the RSE from the regression object.
- The function `sqrt()` computes the square root of a number.

13 Prediction

Finally, we use the regression model to make predictions for new values of the independent variable.

```
# Prediction + prediction interval
tv_new <- 230.1
predicted <- predict(md1, newdata = data.frame(TV = tv_new))
cat("Point estimate at TV =", tv_new, ":", round(predicted, 2), "thousand USD\n")
```

```
## Point estimate at TV = 230.1 : 17.97 thousand USD
```

```
# Approximate 95% prediction interval
cat("95% prediction interval ",
    round(predicted - 2 * RSE_lm, 2), "to",
    round(predicted + 2 * RSE_lm, 2), "thousand USD\n")
```

```
## 95% prediction interval 11.45 to 24.49 thousand USD
```

```
# Exact prediction interval
pred_interval <- predict(md1, newdata = data.frame(TV = tv_new), interval = "prediction",
  ↪ level = 0.95)
cat("Exact 95% prediction interval:",
    round(pred_interval[2], 2), "to",
    round(pred_interval[3], 2), "thousand USD\n")
```

```
## Exact 95% prediction interval: 11.51 to 24.43 thousand USD
```

```
# Confidence interval for the expected value
conf_interval <- predict(md1, newdata = data.frame(TV = tv_new), interval = "confidence",
  ↪ level = 0.95)
cat("95% confidence interval for the expected value:",
    round(conf_interval[2], 2), "to",
    round(conf_interval[3], 2), "thousand USD\n")
```

```
## 95% confidence interval for the expected value: 17.34 to 18.6 thousand USD
```

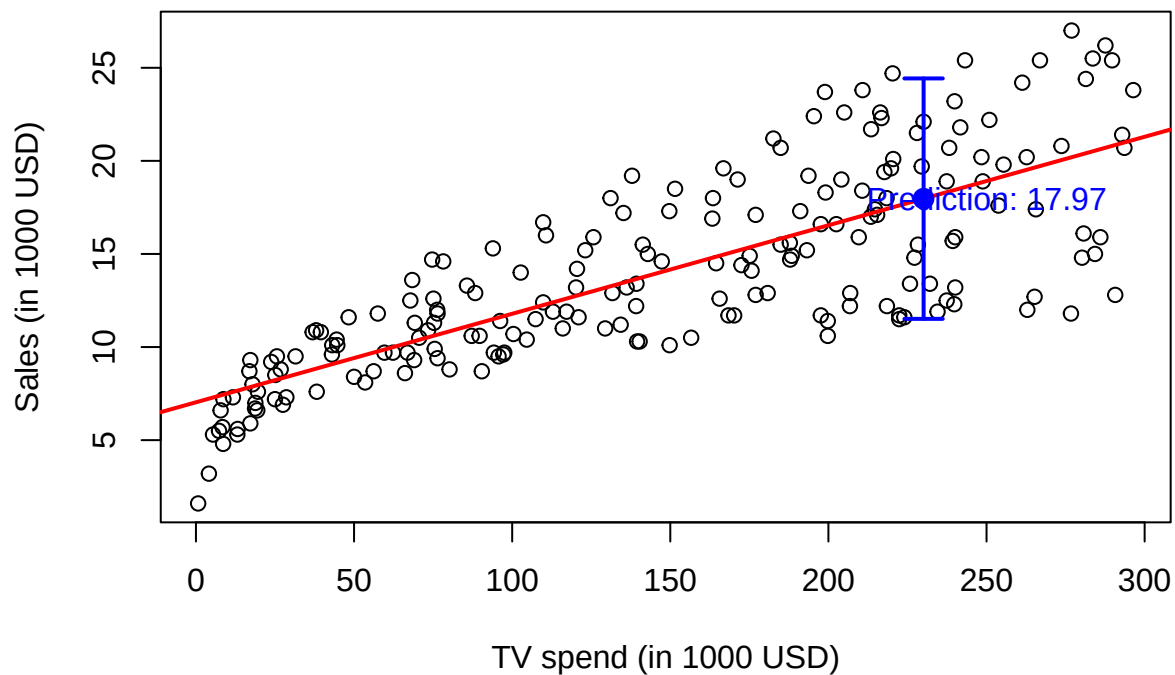
```
# Visualisation of the prediction
plot(sales ~ TV,
```

```

main = "Prediction at TV = 230.1",
xlab = "TV spend (in 1000 USD)",
ylab = "Sales (in 1000 USD)")
abline(md1, col = "red", lwd = 2)
points(tv_new, predicted, col = "blue", pch = 16, cex = 1.5)
arrows(tv_new, pred_interval[2], tv_new, pred_interval[3],
       code = 3, angle = 90, length = 0.1, col = "blue", lwd = 2)
text(tv_new + 20, predicted,
     paste("Prediction:", round(predicted, 2)),
     col = "blue")

```

Prediction at TV = 230.1



```

# Interpretation of the coefficients
cat("\nInterpretation of the coefficients:\n")

```

```

##
## Interpretation of the coefficients:

```

```

cat("- Intercept:", round(coef(md1)[1], 2),
    "\n→ expected sales at TV = 0 (in thousand USD)\n")

```

```

## - Intercept: 7.03 → expected sales at TV = 0 (in thousand USD)

```

```

cat("- Slope:", round(coef(md1)[2], 4),
    "\n→ expected increase in sales per +1 unit of TV (in thousand USD)\n")

```

```

## - Slope: 0.0475 → expected increase in sales per +1 unit of TV (in thousand USD)

```

```
cat(" (i.e. for each additional thousand USD of TV advertising,\n")

## (i.e. for each additional thousand USD of TV advertising,

cat(" expected sales rise by approximately", round(coef(md1)[2] * 1000, 2), "USD)\n")

## expected sales rise by approximately 47.54 USD)
```

Statistical explanation:

- The **prediction** is the predicted value of the dependent variable for a given value of the independent variable: $\hat{y} = b_0 + b_1 x$
- The **prediction interval** specifies the range in which a single new value will lie with a given probability.
- The **confidence interval for the expected value** specifies the range in which the true expected value lies with a given probability.
- Prediction intervals are wider than confidence intervals for the expected value, since they additionally take into account the variation of individual observations around the expected value.
- The **interpretation of the coefficients** is an important aspect of regression analysis:
 - The intercept (b_0) is the expected value of the dependent variable when the independent variable is zero.
 - The slope (b_1) indicates the average change in the dependent variable when the independent variable increases by one unit.

R explanation:

- The function `predict()` with the parameter `interval = "prediction"` computes prediction intervals.
- The function `predict()` with the parameter `interval = "confidence"` computes confidence intervals for the expected value.
- The parameter `level` in `predict()` specifies the confidence level.
- The function `arrows()` draws arrows between two points.
- The function `text()` adds text to a plot.

14 Summary

In this lecture, we introduced simple linear regression, a fundamental statistical method for modelling the relationship between two variables. The key concepts were:

1. **Variation and covariance:** the variation in the data and the relationship between the variables form the basis of regression.
2. **Method of least squares:** this method finds the “best” regression line by minimising the residual sum of squares.
3. **Residuals:** the deviations between observed and predicted values provide information about the quality of the fit.
4. **Decomposition of variation:** the total variation is decomposed into explained and unexplained variation.
5. **Coefficient of determination (R^2):** this measure indicates how well the model explains the variation in the data.
6. **Prediction:** the regression model can be used for forecasts, with prediction intervals quantifying the uncertainty.

Simple linear regression is a powerful tool for data analysis, but it also has limitations. It assumes a linear relationship between the variables and cannot capture more complex relationships. In such cases, more advanced methods such as multiple regression, polynomial regression, or non-parametric procedures may be more appropriate.